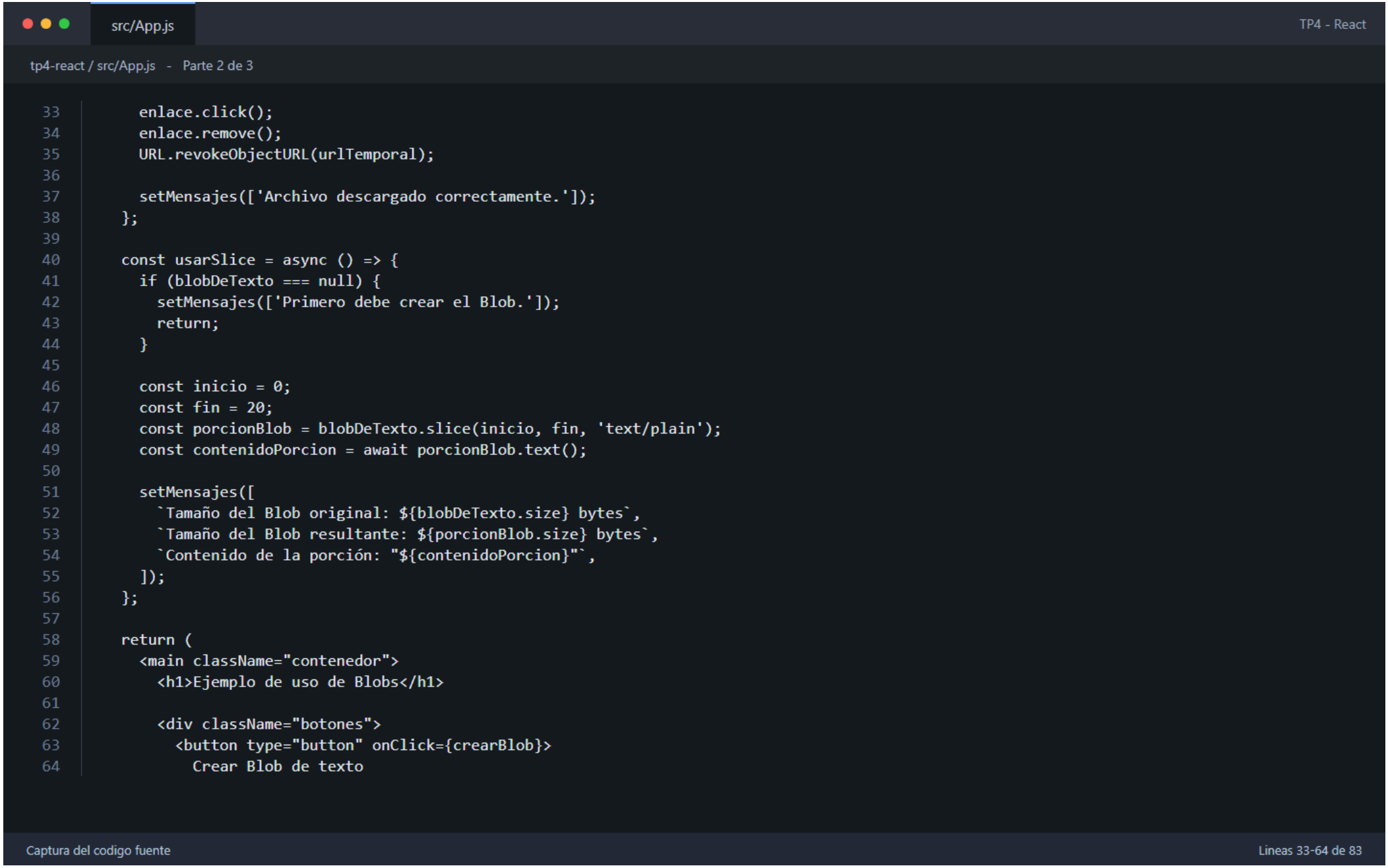
src/App.js TP4 - React

tp4-react / src/App.js - Parte 1 de 3

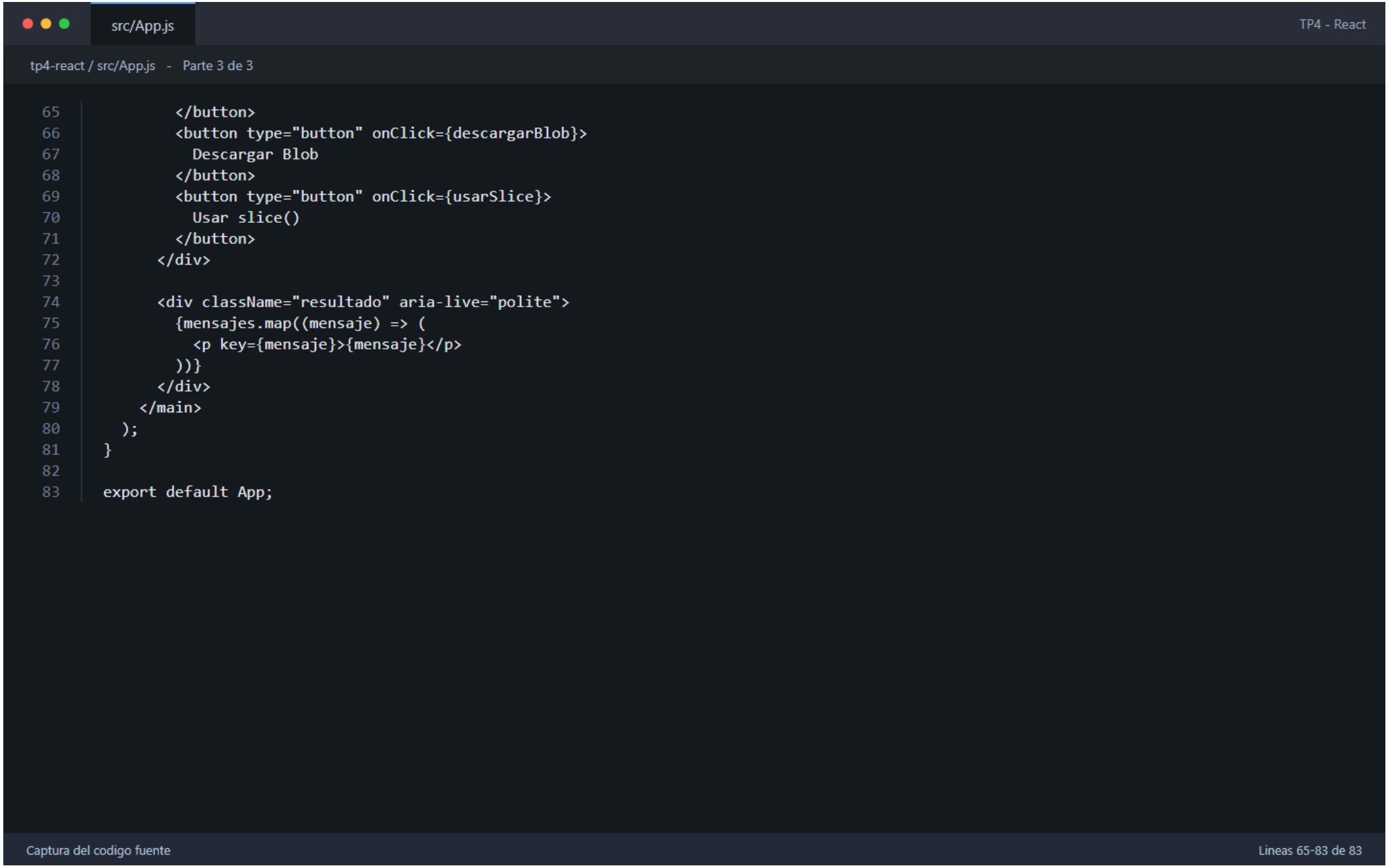
```
1  import { useState } from 'react';
2  import './App.css';
3
4  const TEXTO_ARCHIVO = 'Vamos Central NORTE.';
5
6  function App() {
7    const [blobDeTexto, setBlobDeTexto] = useState(null);
8    const [mensajes, setMensajes] = useState([]);
9
10   const crearBlob = () => {
11     const nuevoBlob = new Blob([TEXTO_ARCHIVO], { type: 'text/plain' });
12
13     setBlobDeTexto(nuevoBlob);
14     setMensajes([
15       'Estado: Blob creado correctamente.',
16       `Tipo: ${nuevoBlob.type}`,
17       `Tamaño: ${nuevoBlob.size} bytes`,
18     ]);
19   };
20
21   const descargarBlob = () => {
22     if (blobDeTexto === null) {
23       setMensajes(['Primero debe crear el Blob.']);
24       return;
25     }
26
27     const urlTemporal = URL.createObjectURL(blobDeTexto);
28     const enlace = document.createElement('a');
29
30     enlace.href = urlTemporal;
31     enlace.download = 'archivo.txt';
32     document.body.appendChild(enlace);
```

Captura del codigo fuente Lineas 1-32 de 83

src/App.js - Parte 1 de 3



src/App.js - Parte 2 de 3



src/App.js - Parte 3 de 3

src/App.css

TP4 - React

tp4-react / src/App.css - Parte 1 de 3

```css
.contenedor {
  width: 100%;
  min-height: 100vh;
  padding: 32px 20px;
  background-color: #f4f4f4;
  text-align: center;
}

h1 {
  margin: 0 0 22px;
  color: #111111;
  font-size: 24px;
  font-weight: 700;
}

.botones {
  display: flex;
  justify-content: center;
  flex-wrap: wrap;
  gap: 16px;
}

button {
  padding: 8px 15px;
  border: 0;
  border-radius: 4px;
  background-color: #4caf50;
  color: #ffffff;
  font-size: 13px;
  cursor: pointer;
}

```

Captura del codigo fuente

Lineas 1-32 de 68

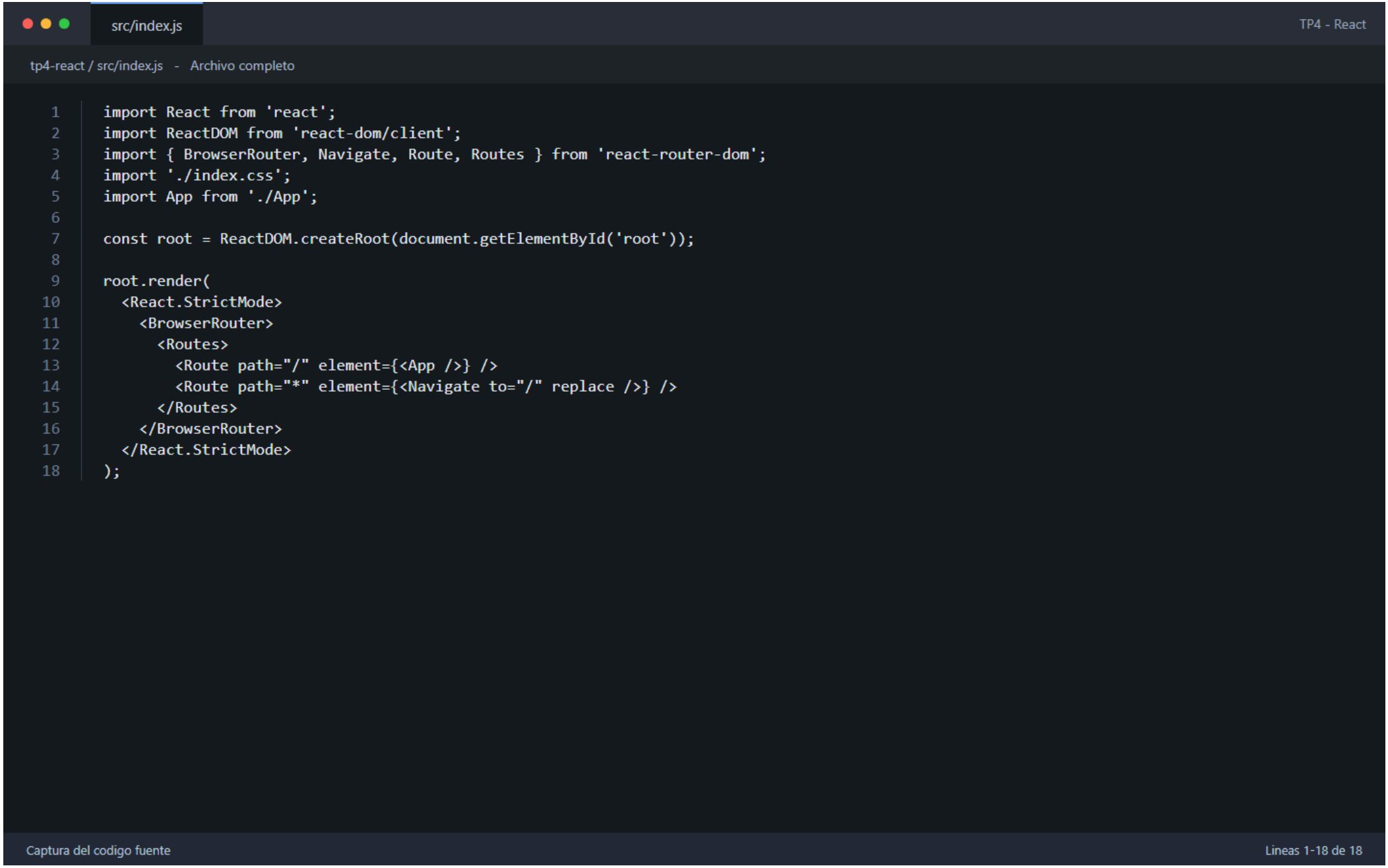
src/App.css - Parte 1 de 3



src/App.css - Parte 2 de 3



src/App.css - Parte 3 de 3


src/index.js
TP4 - React
tp4-react / src/index.js - Archivo completo
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter, Navigate, Route, Routes } from 'react-router-dom';
import './index.css';
import App from './App';
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
<React.StrictMode>
<BrowserRouter>
<Routes>
<Route path="/" element={<App />} />
<Route path="*" element={<Navigate to="/" replace />} />
</Routes>
</BrowserRouter>
</React.StrictMode>
);
Captura del codigo fuente
Lineas 1-18 de 18

src/index.js

src/index.css TP4 - React

tp4-react / src/index.css - Archivo completo

```css
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  background-color: #f4f4f4;
  color: #111111;
  font-family: Arial, Helvetica, sans-serif;
}

#root {
  min-height: 100vh;
}

button,
input,
textarea,
select {
  font: inherit;
}
```

Captura del codigo fuente Lineas 1-22 de 22

src/index.css

src/App.test.js

TP4 - React

tp4-react / src/App.test.js - Archivo completo

```
1  import { fireEvent, render, screen } from '@testing-library/react';
2  import App from './App';
3
4  test('crea un Blob de texto y muestra sus datos', () => {
5    render(<App />);
6
7    fireEvent.click(screen.getByRole('button', { name: /crear blob de texto/i }));
8
9    expect(screen.getByText('Estado: Blob creado correctamente.')).toBeInTheDocument();
10   expect(screen.getByText('Tipo: text/plain')).toBeInTheDocument();
11   expect(screen.getByText(/Tamaño: \d+ bytes/)).toBeInTheDocument();
12 });
13
14 test('avisa si se intenta usar slice antes de crear el Blob', () => {
15   render(<App />);
16
17   fireEvent.click(screen.getByRole('button', { name: /usar slice/i }));
18
19   expect(screen.getByText('Primero debe crear el Blob.')).toBeInTheDocument();
20 });
```

Captura del codigo fuente

Lineas 1-20 de 20

src/App.test.js